

DESARROLLO DE SOFTWARE WEB 2.

EVALUACIÓN NÚMERO 2.

PROFESOR: VICENTE ZAPATA CONCHA.

ALUMNO: IVÁN CAMPOS FARFÁN.

ÍNDICE

- 3. Introducción
- 4. Modelo de Base de Datos
- 5. Código de Entidades (@Entity)
- 6. Código de Repositorios (DAO / @Repository)
- 7. Código de Servicios (Interfaces y @Service)
- 8. Código de Controladores (@Controller)

- 9 Y 10. Funcionamiento del Sistema (Evidencias)
- 11. Conclusión

Link de repositorio en GitHub:

[ivancf89/zudum at zodum-chapter2](https://github.com/ivancf89/zudum-at-zudum-chapter2)

1. Introducción

El presente informe corresponde a la Evaluación Sumativa Unidad 2 del módulo **Desarrollo de Software Web 2**, cuyo objetivo es **implementar la lógica de negocio y la persistencia de datos usando Spring Boot y Spring Data JPA**.

En la **Unidad 1 (EV1)** se trabajó la parte visual mediante Thymeleaf, fragmentos reutilizables (navbar y footer) y vistas de autenticación.

En esta **Unidad 2 (EV2)** se desarrolló el backend completo, conectando la aplicación con una base de datos, incorporando entidades, repositorios, servicios y controladores para lograr operaciones **CRUD reales y funcionales**.

El proyecto elegido se llama **ZUDUM.dev**, una aplicación web de seguimiento de series que permite registrar el progreso de visualización (temporada, episodio y estado) para cada usuario.

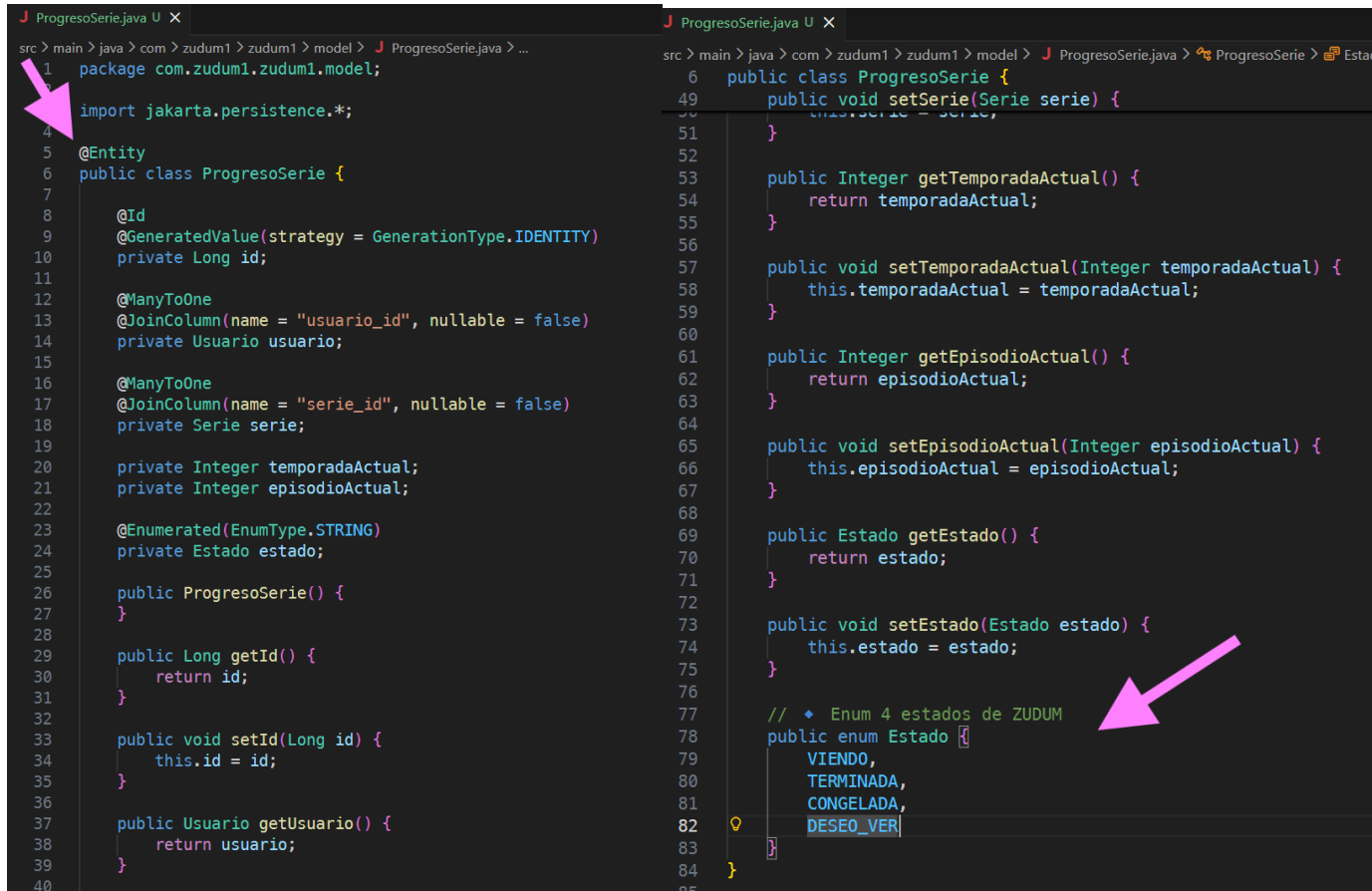
2. Modelo de Base de Datos

La aplicación implementa un modelo relacional compuesto por cinco entidades principales:

Entidad	Función
Usuario	Representa el usuario registrado en la aplicación
Serie	Información base de cada serie disponible
Temporada	Temporadas asociadas a una serie
Episodio	Lista de episodios por temporada
ProgresoSerie	Relación usuario ↔ serie, con estado y progreso

Diagrama resumen de relaciones:

Usuario 1 — N ProgresoSerie N — 1 Serie
Serie 1 — N Temporada 1 — N Episodio



```

src > main > java > com > zudum1 > zudum1 > model > J ProgresoSerie.java > ...
1 package com.zudum1.zudum1.model;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 public class ProgresoSerie {
7
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    @ManyToOne
13    @JoinColumn(name = "usuario_id", nullable = false)
14    private Usuario usuario;
15
16    @ManyToOne
17    @JoinColumn(name = "serie_id", nullable = false)
18    private Serie serie;
19
20    private Integer temporadaActual;
21    private Integer episodioActual;
22
23    @Enumerated(EnumType.STRING)
24    private Estado estado;
25
26    public ProgresoSerie() {
27    }
28
29    public Long getId() {
30        return id;
31    }
32
33    public void setId(Long id) {
34        this.id = id;
35    }
36
37    public Usuario getUsuario() {
38        return usuario;
39    }
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
6 public class ProgresoSerie {
7
8     public void setSerie(Serie serie) {
9         this.serie = serie;
10    }
11
12    public Integer getTemporadaActual() {
13        return temporadaActual;
14    }
15
16    public void setTemporadaActual(Integer temporadaActual) {
17        this.temporadaActual = temporadaActual;
18    }
19
20    public Integer getEpisodioActual() {
21        return episodioActual;
22    }
23
24    public void setEpisodioActual(Integer episodioActual) {
25        this.episodioActual = episodioActual;
26    }
27
28    public Estado getEstado() {
29        return estado;
30    }
31
32    public void setEstado(Estado estado) {
33        this.estado = estado;
34    }
35
36    // * Enum 4 estados de ZUDUM
37    public enum Estado {
38        VIENDO,
39        TERMINADA,
40        CONGELADA,
41        DESEO_VER
42    }
43
44    }

```

3. Capturas del Código de Entidades (@Entity)

En esta sección se documentan las clases del modelo correspondientes a la capa de dominio (Capa del Modelo):

- Usuario.java
- Serie.java
- Temporada.java
- Episodio.java
- ProgresoSerie.java (incluye enum Estado con VIENDO, TERMINADA, CONGELADA, DESEO_VER)
-

```
J ProgresoSerieRepository.java U X
src > main > java > com > zudum1 > zudum1 > repository > J ProgresoSerieRepository.java > ...
1  package com.zudum1.zudum1.repository;
2
3  import com.zudum1.zudum1.model.ProgresoSerie;
4  import org.springframework.data.jpa.repository.JpaRepository;
5
6  import java.util.List;
7  import java.util.Optional;
8
9  public interface ProgresoSerieRepository extends JpaRepository<ProgresoSerie, Long> {
10
11      List<ProgresoSerie> findByUsuarioId(Long usuarioId);
12
13      Optional<ProgresoSerie> findBySerieIdAndUsuarioId(Long serieId, Long usuarioId);
14  }
15
```

4. Capturas del Código de Repositorios (DAO / @Repository)

Interfaces que extienden JpaRepository para abstraer completamente la capa de acceso a datos.

- UsuarioRepository.java
- SerieRepository.java
- TemporadaRepository.java
- EpisodioRepository.java
- ProgresoSerieRepository.java

```
J ProgresoSerieServiceImpl.java 3, U X
src > main > java > com > zudum1 > zudum1 > service > J ProgresoSerieServiceImpl.java > ...

14 @Service
15 public class ProgresoSerieServiceImpl implements ProgresoSerieService {
16
17     private final ProgresoSerieRepository progresoRepo;
18     private final SerieRepository serieRepo;
19     private final UsuarioRepository usuarioRepo;
20
21     public ProgresoSerieServiceImpl(ProgresoSerieRepository progresoRepo,
22                                     SerieRepository serieRepo,
23                                     UsuarioRepository usuarioRepo) {
24         this.progresoRepo = progresoRepo;
25         this.serieRepo = serieRepo;
26         this.usuarioRepo = usuarioRepo;
27     }
28
29     /**
30      * Obtiene el progreso de una serie para un usuario.
31      * Si no existe, crea un registro nuevo con estado VIENDO,
32      * temporada 1, episodio 1.
33      */
34     @Override
35     public ProgresoSerie obtenerProgreso(Long serieId, Long usuarioId) {
36         // 1) Intentar recuperar si ya existe
37         Optional<ProgresoSerie> existente =
38             progresoRepo.findBySerieIdAndUsuarioId(serieId, usuarioId);
39
40         if (existente.isPresent()) {
41             return existente.get();
42         }
43     }
44 }
```

5. Código de Servicios (Interfaces + Implementaciones @Service)

Se presenta la estructura de la capa de negocio, donde se centraliza la lógica para operar con los repositorios.

- Interfaces de servicio
- Implementaciones como ProgresoSerieServiceImpl.java, SerieServiceImpl.java, etc.

```
J ProgresoSerieController.java 5, U X
src > main > java > com > zudum1 > zudum1 > controller > J ProgresoSerieController.java > ...
18
19 @Controller
20 @RequestMapping("/progreso")
21 public class ProgresoSerieController {
22
23     private final SerieRepository serieRepository;
24     private final TemporadaRepository temporadaRepository;
25     private final EpisodioRepository episodioRepository;
26     private final ProgresoSerieRepository progresoSerieRepository;
27     private final UsuarioRepository usuarioRepository;
28
29     public ProgresoSerieController(SerieRepository serieRepository,
30                                   TemporadaRepository temporadaRepository,
31                                   EpisodioRepository episodioRepository,
32                                   ProgresoSerieRepository progresoSerieRepository,
33                                   UsuarioRepository usuarioRepository) {
34         this.serieRepository = serieRepository;
35         this.temporadaRepository = temporadaRepository;
36         this.episodioRepository = episodioRepository;
37         this.progresoSerieRepository = progresoSerieRepository;
38         this.usuarioRepository = usuarioRepository;
39     }
40
```



```
@PostMapping("/guardar")
public String guardarProgreso(
    @RequestParam("serieId") Long serieId,
    @RequestParam("temporadaId") Long temporadaId,
    @RequestParam("episodioId") Long episodioId,
    @RequestParam("estado") String estadoValor,
    Model model
) {
    // Usuario fijo (para la evaluación)
    List<Usuario> usuarios = usuarioRepository.findAll();
    if (usuarios.isEmpty()) {
        model.addAttribute("error", "Debe registrar un usuario antes de guardar el progreso.");
        model.addAttribute("series", serieRepository.findAll());
        return "progreso-form";
    }
}
```

6. Código de Controladores (@Controller)

Controladores responsables de recibir las peticiones HTTP y conectar las vistas con los servicios:

- ApplicationController.java
- CatalogoController.java
- PerfilController.java
- ProgresoSerieController.java

Ejemplos de métodos a mostrar en las capturas:

- Mapeos @GetMapping y @PostMapping
- CRUD completo
- Uso de JSON en selects dependientes (/progreso/temporadas y /progreso/episodios)


```

J ProgresoSerieController.java 5, U X
src > main > java > com > zudum1 > zudum1 > controller > J ProgresoSerieController.java > ...
21 public class ProgresoSerieController {
43     public String mostrarFormularioProgreso(Model model) {
45         return "progreso-form";
46     }
47
48     // ♦ Devuelve temporadas de una serie (JSON para combos dependientes)
49     @GetMapping("/temporadas")
50     @ResponseBody
51     public List<Temporada> obtenerTemporadasPorSerie(@RequestParam("serieId") Long serieId) {
52         return temporadaRepository.findBySerieId(serieId);
53     }
54
55     // ♦ Devuelve episodios de una temporada (JSON para combos dependientes)
56     @GetMapping("/episodios")
57     @ResponseBody
58     public List<Episodio> obtenerEpisodiosPorTemporada(@RequestParam("temporadaId") Long temporadaId) {
59         return episodioRepository.findByTemporadaId(temporadaId);
60     }
61
62     // ♦ Guardar el progreso desde / (registro rápido)
63     @PostMapping("/guardar")
64     public String guardarProgreso(
65         @RequestParam("serieId") Long serieId,
66         @RequestParam("temporadaId") Long temporadaId,
67         @RequestParam("episodioId") Long episodioId,
68         @RequestParam("estado") String estadoValor,
69         Model model
70     ) {

```

7. Funcionamiento del Sistema (Evidencias)

Debe demostrarse visualmente que la persistencia funciona correctamente.

Evidencias recomendadas:

Vista	Acción
/login	Inicio de sesión
/registro	Registro de usuario
/	Catálogo con posters
/	Registro rápido de progreso



Vista	Acción
/perfil	Estado del progreso por usuario
/perfil	Cambio de estado
/perfil	Eliminar de mis series

ZUDUM.dev
Login Registro

Registrar: fast progress bar

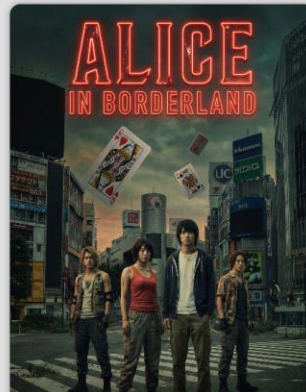
Serie: -- Elige una serie --

Temporada: -- Elige temporada --

Episodio: -- Elige episodio --

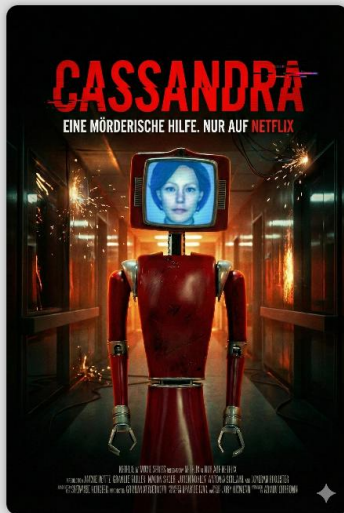
Guardar progreso

Catálogo





Agregar a mi listado



Cassandra

Agregar a mi listado

Agregar a mi listado



Stranger Things

Agregar a mi listado

Agregar a mi listado



Dark

Agregar a mi listado

← ↻ 📄 localhost:8080/perfil

ZUDUM.dev

Login Registro

Mi perfil Zudum

Bienvenido a tu perfil. Aquí puedes ver tu progreso en las series y cambiar el estado o eliminar alguna de tu listado.

Serie	Estado actual	Temporada	Episodio	Cambiar estado		Eliminar
Stranger Things	VIENDO	1	2	Viendo	Actualizar	Eliminar
Emily en París	VIENDO	2	6	Viendo	Actualizar	Eliminar
Dark	TERMINADA	3	8	Terminada	Actualizar	Eliminar
Alice in Borderland	DESEO_VER	1	1	Deseo ver	Actualizar	Eliminar

© 2025 Zudum.dev - En proceso de Desarrollo por Iván

8. Conclusión

La implementación desarrollada permitió conectar correctamente la interfaz construida en EV1 con la base de datos mediante Spring Boot y Spring Data JPA, logrando una arquitectura MVC completa.

Se cumplieron los objetivos de la unidad:

- ✓ Modelar entidades con relaciones JPA
- ✓ Crear repositorios reutilizables mediante JpaRepository
- ✓ Implementar servicios para manejar la lógica de negocio
- ✓ Utilizar controladores que ejecutan operaciones CRUD reales
- ✓ Demostrar persistencia de datos mediante capturas del sistema funcionando

El proyecto se encuentra **completo, funcional y extensible**, permitiendo futuras mejoras como autenticación con roles, favoritos, dark mode y gráficos de progreso.